

Data Structures and Algorithms

Geetha Lekshmy V
Aswathy R Nair
Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

Solving Recurrences - Master Method

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Place your
Webcam Video here
Size 100%

Master Method

- The master method provides bounds for recurrences of the form

$$T(n) = aT(n/b) + f(n) , a>=1,b>1, f(n) \text{ given function}$$

A recurrence of this form characterizes a divide and conquer algorithm that divides a problem of size 'n' into 'a' subproblems each of size n/b and f(n) is the cost of dividing the problem and combining the results of sub problem.

Master Theorem

Master method depends on the following theorem

Given $T(n) = aT(n/b) + f(n)$, $a>=1$, $b>1$, $f(n)$ be a function, $T(n)$ defined on non negative integers, then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a-\varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \theta(n^{\log_b a})$

2. If $f(n) = \theta(n^{\log_b a})$, then $T(n) = \theta(n^{\log_b a} \lg n)$

3. If $f(n) = \Omega(n^{\log_b a+\varepsilon})$, for some constant $\varepsilon>0$ and if $a*f(n/b) <= c*f(n)$ for some constant $c<1$, and all sufficiently large n, then $T(n) =\theta(f(n))$

Master Method - Example

$T(n) = 9T(n/3)+n$

$a=9, b=3, f(n) =n$

$n^{\log_b a} = n^{\log_3 9} = n^2$

Compare $n^{\log_b a}$ and $f(n)$, $n^{2-\varepsilon}$ is polynomially larger than n

$f(n)=O(n^{2-\varepsilon})$, where $\varepsilon =0.5$, So by **case 1** $T(n) =\theta(n^2)$

Master Method - Example

$T(n) = 8T(n/2)+1000n^2$

$a=8, b=2, f(n) = 1000n^2$

$n^{\log_b a} = n^{\log_2 8} = n^3$

Compare $n^{\log_b a}$ and $f(n)$, $n^{3-\varepsilon}$ is polynomially larger than n^2

$f(n)=O(n^{3-\varepsilon})$, where $\varepsilon = 1$, So by **case 1** $T(n) = \theta(n^3)$



Master Method - Example

$T(n) = T(2n/3)+1$

$a=1, b=3/2, f(n) = 1$

$n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$

Compare $n^{\log_b a}$ and $f(n)$, $f(n)$ is $\theta(1)$

So by **case 2** $T(n) = \theta(1 * \lg n) = \theta(\lg n)$

Master Method - Example

$T(n) = 4T(n/2)+n^2$

$a=4, b=2, f(n) = n^2$

$n^{\log_b a} = n^{\log_2 4} = n^2$

Compare $n^{\log_b a}$ and $f(n)$, $f(n)$ is $\theta(n^2)$

So by **case 2** $T(n) = \theta(n^2 * \lg n) = \theta(n^2 \lg n)$

Master Method - Example

$T(n) = 3T(n/4)+n\log n$

$a=3, b=4, f(n) = n\log n$

$n^{\log_b a} = n^{\log_4 3} = n^{0.793}$

Compare $n^{\log_b a}$ and $f(n)$, $f(n)$ is $\Omega(n^{\log_4 3 + \varepsilon})$, $\varepsilon = 0.2$

F(n) is polynomially larger

So **case 3** can be applied, check for regularity condition

$a\, f(n/b) = 3\, f(n/4) = 3\, n/4 \lg(n/4)$

$3\,(n/4)\lg(n/4) <= c\, .n\lg n$ where $c=3/4$

So $T(n) = \theta(n\lg n)$

Master Method - Example

$T(n) = 6T(n/3)+n^2\log n$

$a=6, b=3, f(n) = n^2\log n$

$n^{\log_b a} = n^{\log_3 6} = n^{1.63}$

Compare $n^{\log_b a}$ and $f(n)$, $f(n)$ is $\Omega(n^{1.63+\varepsilon})$, $\varepsilon = 0.2$, $f(n)$ is larger
So **case 3** can be applied, check for regularity condition
$a\, f(n/b) = 6\, f(n/3) = 6\,(n/3)^2 \lg(n/3)$
$6\, n^2/9 \lg(n/3) <= c * n^2\log n$
$2/3 n^2 (\lg n - \lg 3) <= c^* n^2\log n$
$2/3\, n^2 \lg n - 2/3\, n^2 \lg 3 <= c^* n^2\log n \quad c=2/3$
$T(n) = \theta(n^2\lg n)$

Master Method

Polynomially large – ratio of two functions falls between two polynomials.

n^3 is polynomially larger than n^2

$n <= n^3 / n^2 <= n^2$

nlgn is **not polynomially** larger than n

$lgn <= nlgn/n < n$

Not a polynomial

AMRITA | AHEAD Online

Master Method - Example

$T(n) = 2T(n/2)+n\lg n$

$a=2, b=2, f(n) = n\log n$

$n^{\log_b a} = n^{\log_2 2} = n$

Compare $n^{\log_b a}$ and $f(n)$, $f(n)$ is larger, but not polynomialy larger

$f(n)/ n^{\log_b a} = n\log n/n = \log n$
$\log n$ is less than n^{ε} for all positive constants ε

So Master's theorem **can't be** applied here

Master Method

- Master method **cannot** be applied to solve the recurrences in following cases:-

- T(n) is not a monotonic function
  Eg:- T(n) = sin x
- f(n) is not a polynomial
  $T(n) = 3T(n/2) + 2^n$
- b can't be expressed as a constant.
  $T(n) = T(\sqrt{n}) + n$
- f(n) is –ve
  $T(n) = 4T(n/2) - n\log n$

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Summary

Solve recurrences using Master method

Place your
Webcam Video here
Size 100%

Objectives

Implementation in Java

Bubble Sort

Linear Search

Place your
Webcam Video here
Size 100%

Bubble Sort - Algorithm

Data: A set of n integers in an array A, $n \geq 1$
Result: Set of n integers in the array A, arranged in ascending order.

```
i ← 0 ;
while i ≤ n − 2 do
    swapped ← false;
    j ← 0 ;
    while j ≤ n − 2 − i do
        ;                    /* Swap contents of A[j] and A[j+1] */
        if A[j] > A[j + 1] then
            tmp ← A[j];
            A[j] ← A[j + 1];
            A[j + 1] ← tmp;
            swapped ← true;
        end
        j ← j + 1 ;
    end
    ;    /* no swap indicates, array is sorted and loop can be
     terminated */
    if swapped = false then
        break
    end
    i ← i + 1 ;
end
```

Algorithm 1: Bubble Sort

AHEAD Online

Linear Search - Algorithm

Data: A set of n integers in an array A, a number key to be searched in the array

Result: *true* if key is present in the array $false$ otherwise

$i \leftarrow 0$;

while $i < n$ **do**

  if $A[i] = key$ **then**

    return true

  end

  $i \leftarrow i + 1$;

end

return $false$

Algorithm 1: Linear Search

Summary

Implementation of sort and search operations, which are basic array operations.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Place your
Webcam Video here
Size 100%

Objectives

Insertion sort - Analysis

Binary Search - Analysis

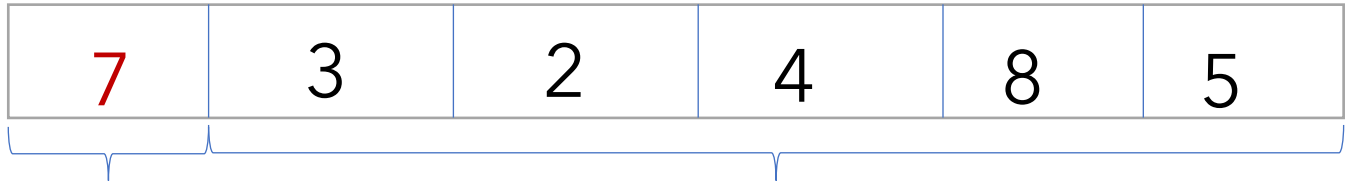
Place your
Webcam Video here
Size 100%

Insertion sort -Algorithm

```
InsertionSort(A)
  for  j=2 to A.length
       k = A[j]
       i=j-1
       while(i>0 && A[i] > k)
                A[i+1]=A[i]
                i=i-1
       A[i+1]=k
```

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Insertion Sort- Example



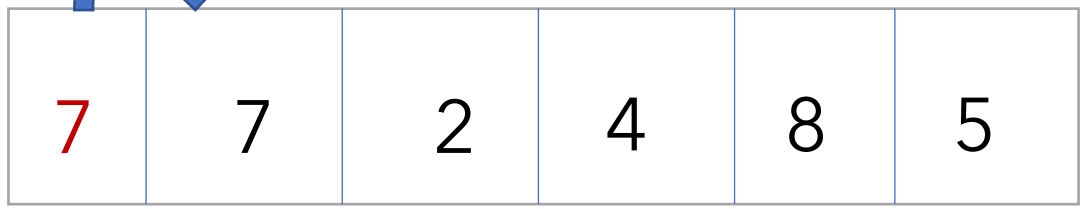
K=3

Sorted

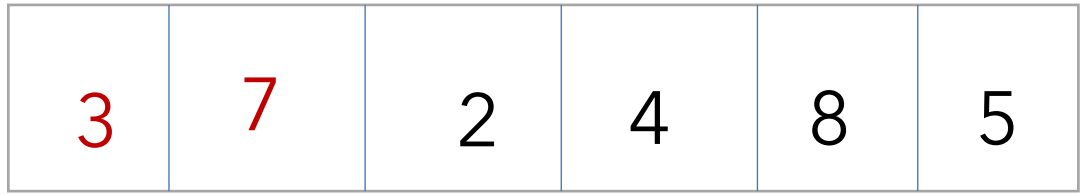
Unsorted

K





K=3

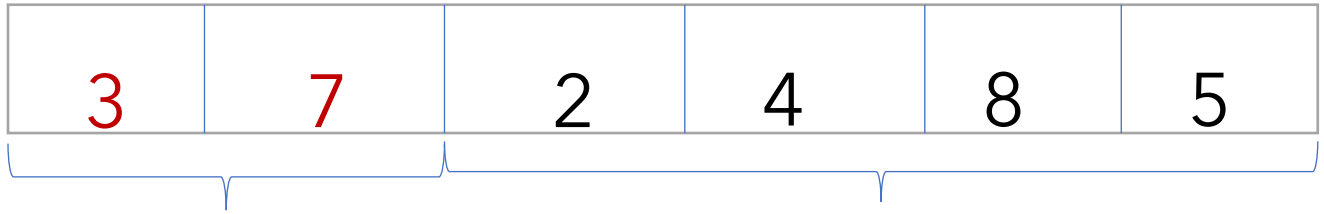


Sorted

Unsorted

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

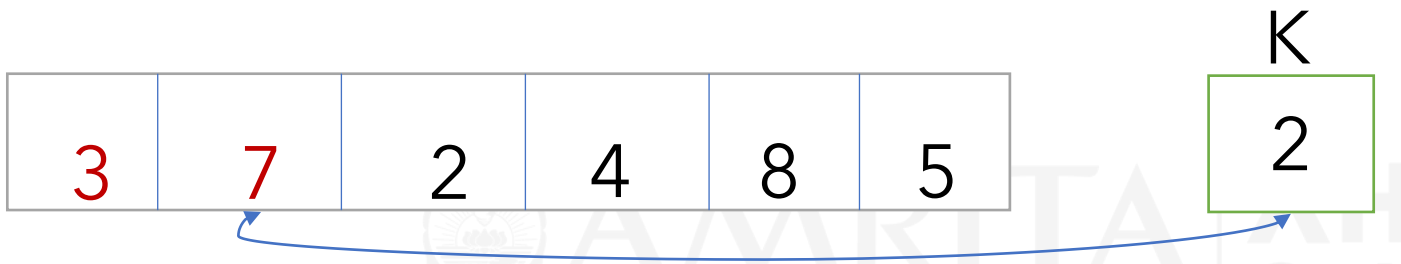
Insertion Sort- Example

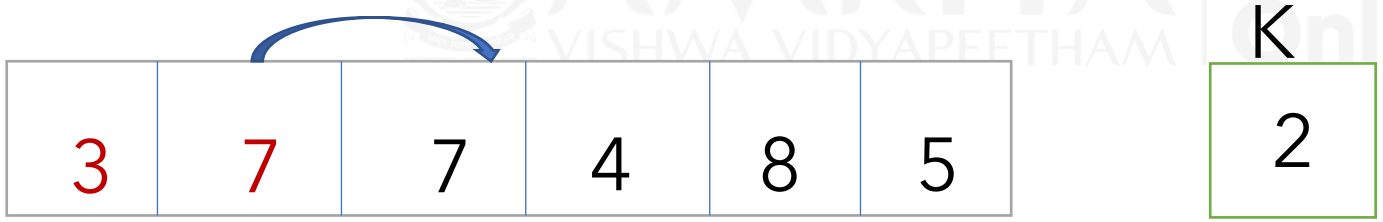


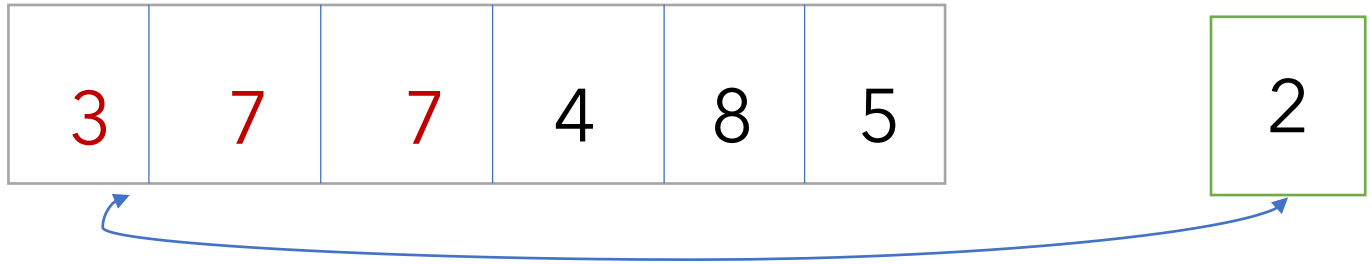
K=2

Sorted

Unsorted

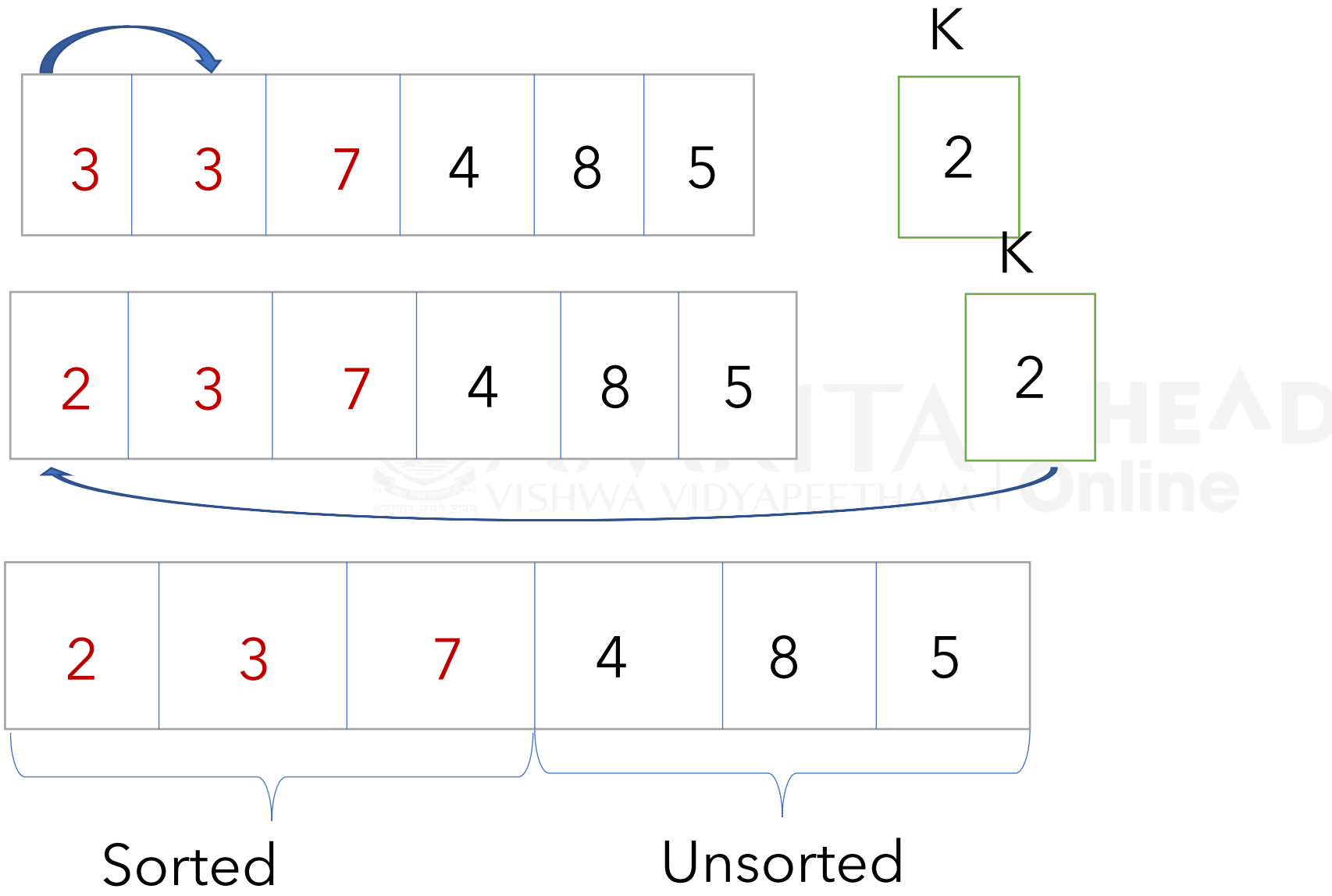






AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Insertion Sort- Example



Insertion sort

```
InsertionSort(A)
  for  j=2 to A.length          → n
      k = A[j]                  →n-1
      i=j-1                     →n-1
      while(i>0 && A[i] > k)
            A[i+1]=A[i]         → j=2 , i=1 ; j=3, i=2,1;j=4, i=3,2,1……
            i=i-1                 j= n,  i= n-1,…1
      A[i+1]=k                    1+2+3+ --+n-1 = n(n-1)/2
```

$T(n) = n+n-1+n-1+n(n-1)/2$

$=3n-2+(n^2- n) / 2$

$=\theta(n^2)$

Insertion sort - When input is sorted

```
InsertionSort(A)
  for  j=2 to A.length          → n
      k = A[j]                  →n-1

      i=j-1                     →n-1
      while(i>0 && A[i] > k)
            A[i+1]=A[i]
            i=i-1
      A[i+1]=k
```

2	3	4	5

$T(n) = n+n-1+n-1$

$= \theta(n)$

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Binary Search

```
bSearch(A[],k,l,h){
  while(l < =  h){
        mid=(l+ h)/2
        if(a[mid]==k)
              return mid
        else if(a[mid]>key)
              h=mid-1
        else
              l=mid+1
        }
}
```

Initial size of array = n

Iteration 1 = n/2

Iteration -2 = n/4

…

Iteration –k – $n/2^k$

When $2^k = n$, loop terminates

$\log(2^k) = \log(n)$

K=log(n)

$T(n) = \theta(\log n)$

Summary

Running time of insertion sort
- Worst case - $\theta(n^2)$
- Best case - $\theta(n)$

Running time of Binary Search - $\theta(\log n)$

Place your
Webcam Video here
Size 100%

References

Introduction to Algorithms – Thomas H Cormen

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

